

Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具

原创 已于 2025-07-25 10:51:53 修改 · 1.3k 阅读 · 29 点赞 · 8 收藏 · CC 4.0 BY-SA版权

文章标签： [#unity](#) [#架构](#) [#c#](#) [#游戏引擎](#) [#开发语言](#)

Unity GC 系列教程第一篇：GC 基础概念与工作原理

Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具

Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）

Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优

Unity GC 系列教程第五篇：高级 GC 内核

欢迎来到 [Unity GC](#) 系列教程的第二篇！在上一篇文章中，我们探讨了 GC 的基本概念和工作原理，理解了 **GC 暂停 (GC Pause)** 和 **GC Alloc** 是导致游戏卡顿的主要原因。现在，是时候将理论付诸实践了！

本篇文章将聚焦于 Unity 开发中那些最常见的导致 **GC Alloc** 的场景。我们会详细分析为什么这些操作会产生 **内存分配**，并引导你使用 Unity 强大的 **Profiler** 工具来识别和定位这些问题。理解这些“坑”，是避免和优化 GC Alloc 的第一步。

2.1 什么是 GC Alloc?

在深入分析具体场景之前，我们先来明确一下 **GC Alloc** 这个概念。

GC Alloc，全称 **Garbage Collection Allocation**，直译过来就是“垃圾回收器内存分配”。它指的是在程序运行时，**向托管堆 (Managed Heap) 申请并分配内存空间的过程**。

当你在 C# 中使用 **new** 关键字创建一个类的实例，或者创建一个数组、字符串等引用类型时，这些操作都会在托管堆上分配内存，从而产生 GC Alloc。

你可能会问：“分配内存不是天经地义的事情吗？为什么它会成为一个问题？”

没错，分配内存是程序运行的必要过程。问题在于：

1. **频繁的分配**：如果你的代码在每一帧或短时间内大量地分配新内存，即便这些内存很快就会被标记为“垃圾”，但频繁的分配行为本身会持续消耗 CPU 资源。
2. **触发 GC**：托管堆的内存使用量是有限的。当堆空间被分配的对象填满，或者达到某个内部阈值时，垃圾回收器就会被触发，进行清理工作。**GC Alloc 越多、越频繁，就越容易触发 GC，进而导致恼人的 GC 暂停。**
3. **内存碎片**：虽然分代 GC 和标记-整理算法 能有效缓解内存碎片，但频繁的小块内存分配和回收仍然可能加剧内存碎片化，影响大对象分配的效率。

所以，我们说的“优化 GC”，很大程度上就是“**减少不必要的 GC Alloc**”。我们的目标是尽可能地重用内存，而不是频繁地创建和销毁对象。

值类型 (Value Types) 与引用类型 (Reference Types)

理解 GC Alloc，必须先区分 C# 中的 **值类型 (Value Types)** 和 **引用类型 (Reference Types)**。

- **值类型**：直接存储数据本身。它们通常在栈 (Stack) 上分配内存，或者作为引用类型对象的一部分直接存储在堆上。当值类型变量超出作用域时，它们的内存会立即被回收，不涉及 GC。
 - **例子**：`int`，`float`，`bool`，`char`，`struct` (结构体)，`enum`。
 - **特点**：按值传递。
- **引用类型**：存储的是对实际数据的引用（内存地址）。它们的数据本身存储在 **托管堆 (Managed Heap)** 上。只有当所有对该对象的引用都消失时，该对象才会被标记为“垃圾”，等待 GC 回收。

- **例子：** `class` (类), `string`, `array` (数组), `delegate`, `interface`。
- **特点：** 按引用传递。

GC Alloc 几乎总是发生在引用类型的内存分配上。 当你在代码中看到 `new` 关键字，或者进行某些操作涉及到引用类型时，就要警惕可能产生 GC Alloc。

2.2 常见的 GC Alloc 场景

了解了 GC Alloc 的本质后，我们来看一下在 Unity/C# 开发中，哪些操作是 GC Alloc 的“高发区”。这些场景往往是新手容易忽视，但对性能影响巨大的地方。

2.2.1 装箱 (Boxing)

装箱 是指将 **值类型隐式或显式地转换为引用类型** 的过程。这个过程中，值类型的数据会被包装到一个新的引用类型对象中，并在托管堆上分配内存。

原理： 当一个值类型需要被当作 `object` 类型（或任何实现了接口的接口类型）来处理时，C# 运行时会为这个值类型创建一个临时的 `object` 对象，将值类型的数据**复制到** 这个新对象中。这个新对象就是在堆上分配的，从而产生了 GC Alloc。

常见场景：

- **字符串拼接：** 当值类型与字符串进行 `+` 拼接时，值类型会被装箱成字符串。

C#

AI运行代码 复制

```
1 // 示例1: int 类型被装箱
2 int score = 100;
3
```

```
3 | string message = "Your score is: " + score; // score 会被装箱
4 |
```

- **非泛型集合**：向 `ArrayList` 或 `Hashtable` 等非泛型集合中添加值类型时。

C#

AI运行代码

复制

```
1 | // 示例2: int 类型被装箱
2 | System.Collections.ArrayList list = new System.Collections.ArrayList();
3 | list.Add(123); // 123 (int) 被装箱成 object
4 |
```

- **通过 `object` 类型传递值类型参数**：当方法的参数是 `object` 类型，而你传入了一个值类型时。

C#

AI运行代码

复制

```
1 | // 示例3: int 类型被装箱
2 | void LogObject(object obj)
3 | {
4 |     Debug.Log(obj);
5 | }
6 | // 调用时
7 | int health = 50;
8 | LogObject(health); // health 会被装箱
9 |
```

- **Enum 类型与 `ToString()`**：尽管 `Enum` 是值类型，但其 `ToString()` 方法在内部可能触发装箱。

为什么重要？ 尽管单个装箱操作的开销很小，但如果在游戏循环（如 `Update` 或 `LateUpdate`）中频繁发生，累积起来就会产生巨大的 GC Alloc，迅速填满堆内存，导致 GC 频繁触发。

2.2.2 字符串操作 (String Operations)

string 是引用类型，并且是不可变 (Immutable) 的。 这意味着一旦一个字符串被创建，它的内容就不能被修改。任何看起来修改字符串的操作（如拼接、截取、替换）实际上都会创建一个新的字符串对象，并将旧字符串的内容复制到新字符串中。

原理：字符串的不可变性是为了安全和效率考虑（例如，字符串可以被哈希并作为 Dictionary 的键）。但这也意味着，每次“修改”字符串都会产生新的 GC Alloc。

常见场景：

- **字符串拼接 + :**

C#

AI运行代码 复制

```
1 // 示例1: 连续拼接会导致多个临时字符串对象
2 string name = "Player";
3 int level = 5;
4 string info = "Name: " + name + ", Level: " + level.ToString() + "!";
5 // 这里可能会创建至少 3 个临时字符串对象
6
```

- **string.Format() / 插值字符串 \$:** 虽然它们提供了更优雅的语法，但本质上也是创建新字符串。

C#

AI运行代码

复制

```
1 // 示例2: 同样会产生新的字符串对象
2 string formattedInfo = string.Format("Name: {0}, Level: {1}!", name, level);
3 string interpolatedInfo = $"Name: {name}, Level: {level}!";
4
```

- **Substring() / Split() / Replace() 等方法:**

C#

AI运行代码

复制

```
1 // 示例3: 生成新的字符串对象
2 string original = "Hello World";
3 string sub = original.Substring(0, 5); // "Hello"
4 string[] parts = original.Split(' '); // {"Hello", "World"}
5
```

解决方案方向: 对于频繁的字符串拼接, 应该使用 **System.Text.StringBuilder** 类。它提供了可变的字符串操作, 只在最后调用 **ToString()** 时才产生一次 GC Alloc。

2.2.3 集合类型操作 (Collection Operations)

某些集合操作或迭代方式会产生临时的数组或枚举器对象。

原理: 为了提供某些功能或迭代机制, C# 运行时或 Unity API 会在后台创建新的引用类型对象。

常见场景:

- **List<T>.ToArray() / List<T>.ToList() :**

C#

AI运行代码

复制

```
1 // 示例1: 每次调用都会创建一个新的数组/List
2 List<int> numbers = new List<int>() { 1, 2, 3 };
3 int[] numArr = numbers.ToArray(); // 新数组分配
4 List<int> numList = numArr.ToList(); // 新 List 分配
5
```

这些方法会复制集合中的所有元素到一个新的数组或列表中，从而产生 GC Alloc。

- **Dictionary<TKey, TValue>.Keys / Values :**

C#

AI运行代码

复制

```
1 // 示例2: 每次访问 Keys 或 Values 属性都会创建一个新的集合
2 Dictionary<string, int> scores = new Dictionary<string, int>() { { "A", 1 }, { "B", 2 } };
3 foreach (string key in scores.Keys) // 每次循环都可能创建 Dictionary.KeyCollection 的临时对象
4 {
5     Debug.Log(key);
6 }
7
```

在旧版本的 .NET 或 Mono 中，每次访问 **Keys** 或 **Values** 属性都会创建一个新的集合对象。即便是在较新版本中，虽然返回的是视图而非新集合，但在 **foreach** 循环中使用时，其内部的枚举器仍然可能产生 GC Alloc。

- **foreach 循环 (对某些非集合类型):**

当对某些不是 Array 或 List 的可枚举类型（实现了 **IEnumerable** 接口但没有 **GetEnumerator** 方法且其返回值没有 **Current** 和 **MoveNext** 的结构体）进行 **foreach** 循环时，编译器为了实现迭代，可能会生成一个临时的枚举器 (Enumerator) 引用类型对象。

C#

AI运行代码

复制

```
1 // 示例3: 自定义可枚举类型, 可能导致枚举器装箱或分配
2 public struct MyEnumerable : IEnumerable<int>
3 {
4     public IEnumerator<int> GetEnumerator() => new MyEnumerator(); // 假设 MyEnumerator 是一个类
5     IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
6 }
7
8 public class MyEnumerator : IEnumerator<int> // 如果是类, 就会分配
9 {
```

展开 ∨

优化方向：对于 `List<T>.ToArray()` 等操作，如果结果仅用于读取且不修改，可以考虑使用索引或传入预分配的数组。对于 `Dictionary` 的 `Keys/Values`，可以缓存。对于 `foreach` 循环，优先使用 `for` 循环（当索引可用时），或确保自定义迭代器是值类型（`struct`）。

2.2.4 Unity API 调用

Unity 引擎 的某些 API 为了方便使用，可能会在内部创建新的数组或对象。这在游戏循环中频繁调用时会成为 GC Alloc 的大户。

原理：Unity 引擎在 C++ 层实现，通过 P/Invoke (Platform Invoke) 与 C# 层交互。有些 API 设计上为了简化调用者的代码，会在 C# 层返回新的托管对象。

常见场景：

- `GameObject.GetComponent<T>()` / `GameObject.GetComponents<T>()` :

- `GetComponent<T>()` 通常是安全的，返回现有组件的引用。
- `GetComponents<T>()` / `GetComponentsInChildren<T>()`：这些方法会返回一个新的 `T[]` 数组。

C#

AI运行代码

复制

```
1 // 示例1: 每次调用都会创建一个新的组件数组
2 MeshFilter[] filters = GetComponentsInChildren<MeshFilter>();
3 foreach (var filter in filters)
4 {
5     // ...
6 }
7
```

优化方向：对于 `GetComponents` 类方法，使用 `GetComponentsNonAlloc<T>()` (Unity 2021+) 或传入预分配的 `List<T>` (例如 `GetComponents(list)`)。

- **`Camera.allCameras` / `Camera.main`：**

- **`Camera.allCameras`**：每次调用都会创建一个新的 `Camera[]` 数组，包含场景中所有激活的相机。

C#

AI运行代码

复制

```
1 // 示例2: 每帧调用会导致频繁的数组分配
2 foreach (Camera cam in Camera.allCameras)
3 {
4     // ...
5 }
6
```

}

- Camera.main：在大多数情况下，第一次调用会查找并缓存主相机，后续调用会返回缓存的引用，但如果场景中主相机频繁变化（比如加载新场景），或者你没有设置 Tag 为 “MainCamera”，它每次查找仍然会产生开销。

优化方向：在 Awake() 或 Start() 中缓存 Camera.main 的引用，避免在 Update() 中频繁调用 Camera.allCameras。

- 物理查询 (Physics Queries):

Physics.RaycastAll、Physics.OverlapSphere 等方法会返回一个新的 RaycastHit[] 或 Collider[] 数组。

C#

AI运行代码 复制

```
1 // 示例3: 在 Update 中进行频繁物理查询
2 RaycastHit[] hits = Physics.RaycastAll(transform.position, Vector3.forward);
3 foreach (var hit in hits)
4 {
5     // ...
6 }
7
```

优化方向：使用 **Physics.RaycastNonAlloc**、**Physics.OverlapSphereNonAlloc** 等 **NonAlloc** 变体，它们允许你传入一个预分配的数组来存储结果，避免新的内存分配。

- Input.touches / Input.GetTouch():

Input.touches 返回一个 Touch[] 数组。Input.GetTouch() 也可能涉及内部数组的拷贝。

优化方向：可以自己维护一个 List，并根据 Input.touchCount 和 Input.GetTouch(index) 来获取数据，避免直接使用 Input.touches。

- 协程 (Coroutines) 中的 `yield return new WaitForSeconds()` 等：

每次 `yield return new WaitForSeconds(time)` 都会创建一个新的 `WaitForSeconds` 对象。

C#

AI运行代码复制

```
1 // 示例4：每次循环都会分配新的 WaitForSeconds 对象
2 IEnumerator MyCoroutine()
3 {
4     while (true)
5     {
6         yield return new WaitForSeconds(1.0f); // 每次都会 new
7         Debug.Log("Waited one second.");
8     }
9 }
```

展开

优化方向：将 `new WaitForSeconds(time)` 的结果缓存起来，重用同一个对象。

2.2.5 匿名函数与 Lambda 表达式捕获变量 (Closures)

当 **匿名方法** 或 **Lambda 表达式** 捕获 (Capture) 了外部作用域的变量时，编译器会为这些被捕获的变量生成一个 **闭包 (Closure)** 对象。这个闭包对象是引用类型，并在堆上分配内存。

原理：为了让匿名函数或 Lambda 表达式在它被定义的作用域之外（甚至在定义它的方法返回之后）仍然能访问到那些变量，编译器需要把这些变量“提升”到一个新的类中。这个新类的实例就是闭包。

常见场景：

C#

AI运行代码

复制

```
1 // 示例1: Lambda 表达式捕获局部变量
2 public void StartGame(int initialScore)
3 {
4     // 这里捕获了 initialScore
5     UnityEditor.EditorApplication.delayCall += () =>
6     {
7         Debug.Log("Game started with score: " + initialScore);
8     }; // 这个 Lambda 表达式会产生闭包对象
9 }
```

展开 ∨

`StartGame` 方法中的 Lambda 表达式捕获了 `initialScore`，因此会生成一个匿名类的实例来存储 `initialScore`，这个实例就是 GC Alloc。在 `CreateActions` 示例中，更糟糕的是，由于 `i` 是循环变量，如果不加注意，所有闭包都将引用同一个 `i` 的最终值。

优化方向：避免在热路径中（尤其是在 `Update` 循环或频繁调用的方法中）使用捕获外部变量的 Lambda 表达式。如果必须捕获，考虑将捕获的变量作为参数传递给一个普通方法，或者确保 Lambda 不在性能敏感的地方频繁创建。

2.2.6 LINQ (Language Integrated Query)

LINQ 提供了一种强大的、声明式的查询语法，让集合操作变得简洁。然而，**大多数 LINQ 查询都会在内部创建临时的集合或枚举器，导致大量的 GC Alloc。**

原理：LINQ 的很多扩展方法（如 `Where()`，`Select()`，`OrderBy()`，`ToList()`，`ToArray()` 等）都是惰性求值 (Lazy Evaluation) 的，它们返回的通常是新的可枚举对象或迭代器。当这些查询最终被执行（例如，通过 `foreach` 或 `ToList()`）时，就会在堆上分配内存。

常见场景：

C#

AI运行代码 复制

```
1 // 示例1: 筛选并转换为 List
2 List<Enemy> enemies = GetActiveEnemies();
3 List<Enemy> deadEnemies = enemies.Where(e => e.IsDead).ToList(); // Where 和 ToList 都会产生 GC Alloc
4
5 // 示例2: 排序
6 int[] numbers = new int[] { 3, 1, 4, 1, 5 };
7 IEnumerable<int> sortedNumbers = numbers.OrderBy(n => n); // OrderBy 也会产生临时对象
8
9 // 示例3: 链式查询
```

展开

为什么重要？ LINQ 的语法糖非常诱人，但它的底层实现通常涉及多个临时的集合或迭代器分配。在非性能敏感的地方使用 LINQ 是可以接受的，但在游戏的核心循环、更新逻辑或任何需要高帧率的地方，**强烈建议避免使用 LINQ。**

优化方向：手动实现 LINQ 类似的逻辑，使用 `for` 循环、预分配集合等方式来避免 GC Alloc。

2.3 如何分析 GC Alloc? Unity Profiler 深度解析

现在你已经对常见的 GC Alloc 场景有所了解，但仅仅知道这些是远远不够的。在实际项目中，你可能会发现代码中存在大量隐藏的 GC Alloc，而 Unity 提供了强大的 **Profiler** 工具来帮助你发现并定位它们。

2.3.1 Unity Profiler (CPU Usage 模块)

Unity Profiler 是你进行性能优化的第一把也是最重要的一把工具。在 Unity 编辑器中，你可以通过 `Window > Analysis > Profiler` 打开它。

1. 连接到设备：

- 在编辑器中运行时，Profiler 会自动连接到当前运行的播放器。
- 如果要分析真机上的性能，你需要将应用程序构建为 Development Build，并在 Player Settings 中勾选 “Autoconnect Profiler”。然后通过 Profiler 窗口左上角的下拉菜单选择你的设备。

2. 选择 CPU Usage 模块：

在 Profiler 窗口顶部，确保你选择了 “CPU Usage” 模块。这个模块会详细显示每一帧 CPU 耗时的情况。

3. 识别 **GC.Alloc** 条目：

- 在 CPU Usage 模块的 **Hierarchy (层级)** 视图中，你可以看到每一帧的函数调用栈和耗时。
- 重点关注一个名为 **GC.Alloc** 的条目。这个条目直接显示了在该帧中发生的总内存分配量（以字节为单位）。
- 当你看到 **GC.Alloc** 的值持续很高时（例如，每帧几 KB 甚至几十 KB 以上），这就表明你的游戏正在频繁地进行内存分配，很可能导致 GC 频繁触发和卡顿。

(这是一个示意图，实际界面可能略有不同。注意箭头指向的 **GC.Alloc**)

4. 定位 GC Alloc 的来源：

- 点击 **GC.Alloc** 条目，然后查看其下方的 **Call Stack (调用堆栈)** 面板。这个面板会显示是哪行代码导致了内存分配。
- 通常，你会看到类似 `string.Concat`、`List<T>.ToArray`、或者某个 Unity API 调用（如 `Physics.RaycastAll`）出现在调用堆栈中。
- 如果 Call Stack 显示的是 `new XXX()`，那么你就直接找到了分配源头。

小技巧：在 Profiler 窗口的 Hierarchy 视图中，你可以点击右上角的 “**Columns**” (列) 按钮，勾选 “**GC Alloc**” 列。这样，每一项函数调用的右侧都会直接显示它内部产生的 GC Alloc 量，这对于快速定位问题非常有用。

5. 帧率曲线与 GC Spikes (尖峰):

- 在 Profiler 窗口的顶部, 你可以看到帧率曲线 (FPS) 和 CPU Usage 曲线。
- 当发生 GC 时, 你通常会在 CPU Usage 曲线中看到一个明显的 **尖峰 (Spike)**, 同时帧率曲线会骤然下跌。这就是 GC 暂停的直观表现。
- 通过观察这些尖峰, 你可以回溯到尖峰发生的那一帧, 然后深入分析该帧的 `GC.Alloc` 和调用堆栈。

2.3.2 Memory Profiler (内存分析工具)

虽然 CPU Usage 模块能告诉你“在哪里”发生了 GC Alloc, 但 **Memory Profiler (内存分析器)** 能更详细地告诉你“**什么**”对象占据了内存, “**谁**”引用了它们, 以及 **GC Roots 是什么**。

注意: Memory Profiler 是一个单独的包, 你可能需要在 Unity Package Manager 中安装它 (Window > Package Manager, 选择 `Unity Registry`, 搜索 `Memory Profiler`)。

1. 打开 Memory Profiler: Window > Analysis > Memory Profiler。

2. 捕获快照 (Capture Snapshot):

- 连接到你的游戏运行实例。
- 点击 Memory Profiler 窗口中的“**Capture Player**”或“**Capture Editor**”按钮来捕获当前的内存快照。
- 通常, 你会捕获两个快照: 一个在你的游戏场景正常运行一段时间后, 另一个在发生可疑的内存增长 (例如, 加载了新场景、触发了某个功能后) 之后。

3. 分析快照数据:

- Memory Profiler 会显示多个视图，包括 **“All Objects”** (所有对象)、**“Managed Objects”** (托管对象)、**“Native Objects”** (原生对象) 等。
- **“Managed Objects”** 视图是我们关注的重点。它会列出所有托管堆上的对象，以及它们的大小。
- 你可以按 **Size (大小)** 或 **Count (数量)** 排序，找出那些占用内存最大或数量最多的对象。
- **查找 GC Roots**：点击某个可疑对象，Memory Profiler 会显示它的 **“References” (引用者)** 和 **“Referenced By” (被引用者)** 视图。通过追溯 “References” 链，你可以找到是哪个 GC Root 最终引用了这个对象，从而判断它是否真的无法被回收（内存泄漏）。
- **比较快照**：Memory Profiler 最强大的功能之一是 **比较两个快照**。通过比较游戏正常运行前后的快照，你可以快速找出哪些对象在持续增长，从而定位潜在的内存泄漏或持续的 GC Alloc 问题。

(这是一个示意图，实际界面可能略有不同。注意左侧的分类和右侧的详细信息)

Memory Profiler 的使用场景：

- **发现内存泄漏**：如果某个对象在你认为它应该被销毁后仍然存在，Memory Profiler 可以帮你找出是哪个引用阻止了它被 GC。
- **分析大对象**：找出哪些自定义类、数组或资产在内存中占据了大量空间。
- **确认 GC Alloc 问题**：通过观察对象数量和大小的变化，验证你的 GC Alloc 优化是否有效。

2.3.3 Profiler 的 “GC Alloc” 列

这是 Profiler 中一个非常实用的功能，在 CPU Usage 模块中，你可以自定义显示列。

1. 打开 Profiler，进入 CPU Usage 模块。

2. 点击 Hierarchy 视图右上角的 **“Columns”** 按钮。

3. 在弹出的菜单中，勾选 **“GC Alloc”**。

4. 现在，在 Hierarchy 视图中，每一行（每个函数调用）的右侧都会多一列，显示该函数调用所产生的 GC Alloc 量。这使得你可以一目了然地看到哪些方法是 GC Alloc 的主要贡献者。

这比只看 `GC.Alloc` 总量然后深入 Call Stack 更高效，它能直接告诉你每个函数调用的“污染”程度。

总结

在本篇教程中，我们深入理解了 **GC Alloc** 的概念，它是我们优化 GC 的核心目标。我们详细列举了 Unity 开发中最常见的导致 GC Alloc 的场景，包括：

- **装箱 (Boxing)**：值类型到引用类型的转换。
- **字符串操作 (String Operations)**：字符串的不可变性导致的频繁新对象创建。
- **集合类型操作 (Collection Operations)**：`ToArray()`，`Keys`，`foreach` 迭代器等产生的临时对象。
- **Unity API 调用**：某些便捷 API 在内部创建新的数组或对象。
- **匿名函数与 Lambda 表达式捕获变量 (Closures)**：闭包对象的分配。
- **LINQ**：其便利性背后隐藏的额外内存分配。

我们还详细介绍了如何利用 Unity 自带的强大工具——**Unity Profiler**（特别是 CPU Usage 模块的 `GC.Alloc` 条目和 GC Alloc 列）和 **Memory Profiler** 来识别、定位和分析这些 GC Alloc 问题。掌握这些工具的使用，是你在实际项目中进行性能优化的必备技能。

在下一篇中，我们将开始进入实战环节，学习具体的代码优化技巧，来减少这些常见的 GC Alloc，从而提升游戏的流畅性。做好准备，我们将在代码中见真章！

[Unity GC 系列教程第一篇：GC 基础概念与工作原理](#)

[Unity GC 系列教程第二篇：Unity 中常见的 GC Alloc 场景与分析工具](#)

[Unity GC 系列教程第三篇：GC Alloc 优化技巧与实践（上）](#)

[Unity GC 系列教程第四篇：GC Alloc 优化技巧与实践（下）与 GC 调优](#)

[Unity GC 系列教程第五篇：高级 GC 内核](#)

[关于我们](#) [招贤纳士](#) [商务合作](#) [寻求报道](#) ☎ 400-660-0108 ✉ kefu@csdn.net 🗣 [在线客服](#) 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 [经营性网站备案信息](#) [北京互联网违法和不良信息举报中心](#)
[家长监护](#) [网络110报警服务](#) [中国互联网举报中心](#) [Chrome商店下载](#) [账号管理规范](#) [版权与免责声明](#) [版权申诉](#) [出版物许可证](#) [营业执照](#)

©1999-2025北京创新乐知网络技术有限公司